3. Tags, Follows und Likes

Natürlich gibt es noch ein paar weitere Funktionen in YourSozialNet, die du auch von anderen Netzwerken kennst. Alle drei sind **n-zu-m-Kardinalitäten**, weswegen – wie bei den Kursen in Kapitel I – Hilfstabellen benötigt werden.

- **tags**: Ein Foto kann viele Tags haben (#kiel, #segeln), und ein Tag steht an vielen Fotos. Die Klasse **tags** speichert jeden Tag nur einmal, die Hilfstabelle **photo_tag** verknüpft Fotos und Tags. Das ist die „klassische“ n-zu-m-Beziehung.
- **follows**: Ein User folgt vielen Usern und hat viele Follower. Hier verbindet die Hilfstabelle die Klasse users **mit sich selbst**: `follower_id ↑` (wer folgt) und `following_id ↑` (wem gefolgt wird) zeigen beide auf users.
- **likes**: Ein User liked viele Fotos, ein Foto wird von vielen Usern gelikt. Die Hilfstabelle bekommt ein zusätzliches Attribut **kind** – die Art des Likes (heart, thumb, laugh).

Bei den drei Hilfstabellen ist der Primärschlüssel **zusammengesetzt** (siehe I.2.1): `(photo_id, tag_id)`, `(follower_id, following_id)` und `(user_id, photo_id)`. So kann niemand ein Foto zweimal liken.

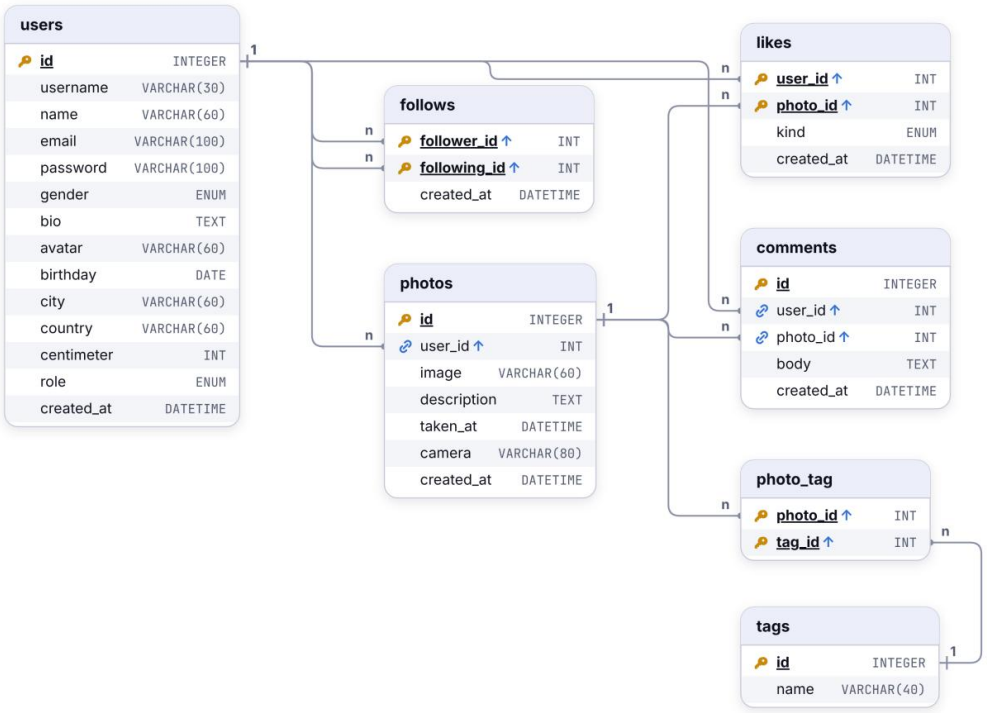


Abb. 9: Das Netzwerk mit Tags, Follows und Likes

JOIN über eine Hilfstabelle

Um eine n-zu-m-Beziehung auszugeben, geht man über die Hilfstabelle: von der einen Klasse zur Hilfstabelle und von dort zur anderen Klasse. Es werden also drei Tabellen verbunden.

AUFBAU

```
SELECT <Liste von Attributen>
FROM <Klasse1>
JOIN <Hilfstabelle>
  ON <Fremdschlüssel1> =
<Primärschlüssel1>
JOIN <Klasse2>
  ON <Primärschlüssel2> =
<Fremdschlüssel2>
WHERE <Bedingung>
```

BEISPIEL · ALLE TAGS DES FOTOS MIT DER OSTSEE (SEASCAPE)

```
SELECT photos.image, tags.name
FROM photos
JOIN photo_tag ON photo_tag.photo_id = photos.id
JOIN tags      ON tags.id = photo_tag.tag_id
WHERE photos.image = 'dummy/linus_seascape.webp'
```

Übrigens: comments funktioniert genauso. Ein Kommentar verbindet einen User mit einem Foto – wer von users über comments zu photos will, braucht also auch zwei JOINs.

KOMMENTARE MIT VERFASSER UND FOTO

```
SELECT users.username, comments.body, photos.image
FROM users
JOIN comments ON comments.user_id = users.id
JOIN photos   ON comments.photo_id = photos.id
```

ALLE USER, DIE DAS FOTO MIT DER OSTSEE GELIKT HABEN – UND WIE

```
SELECT users.username, likes.kind
FROM users
JOIN likes  ON likes.user_id = users.id
JOIN photos ON likes.photo_id = photos.id
WHERE photos.image = 'dummy/linus_seascape.webp'
```

Eine Klasse zweimal: Aliasse

Bei follows braucht man die Klasse users **zweimal**: einmal für den, der folgt, und einmal für den, dem gefolgt wird. Damit SQL die beiden auseinanderhalten kann, bekommt jede einen eigenen Namen, einen **Alias**: `users AS fan` und `users AS star`.

WEM FOLGT LEVI.KICKS?

```
SELECT fan.username AS folgt, star.username AS wem
FROM follows
JOIN users AS fan  ON follows.follower_id  = fan.id
JOIN users AS star ON follows.following_id = star.id
WHERE fan.username = 'levi.kicks'
```

Aufgabe 6

Notiere für die folgenden Abfragen einen SQL-Befehl.

a) Unter welchen Bildern stehen die Echsen-Kommentare aus Aufgabe 5 h)? Gib Benutzername, Kommentar und Bild aus.

b) ★ Welches Foto hat die meisten Kommentare – und wem gehört es? *Achtung: Hier gibt es zwei verschiedene User – den Verfasser des Kommentars und den Besitzer des Fotos.*

c) Gib eine Tabelle aus, in der alle Fotos zusammen mit ihren Tags stehen.

d) Wie viele Bilder haben den Tag „abenteuerlust“? Vergleiche dein Ergebnis mit `LIKE '%abenteuer%'`. Welche Zahl ist richtig?

e) Welche Tags hat Alina Keller genutzt?

f) Gib die 10 beliebtesten Fotos aus (die meisten Likes) – mit dem Benutzernamen des Besitzers.

g) Erinnerst du dich an Erna aus Aufgabe 3 j)? Wem folgt sie? Ist ihr Enkel dabei?

h) ★ Welche User folgen sich gegenseitig?

★ zum Knobeln